Vending Machine Application

Brief:

For this assignment I have been asked to create a vending machine application using the C++ programming language. This program demonstrates my ability of using the C++ language through what I have learned in lesson and additional learning I have done in my spare time. I have used the raylib library to create a user interface with pixel art images that I have drawn.
The following techniques are used throughout my program:
Multiple Files, Classes, Functions, Loops, Vectors, Arrays, Switch Statements, If-Statements.

Link to my Github: https://github.com/CodeLab-I-21-22/utility-app-SolomonMoulangLewis/tree/main/Utility%20App/Vending%20Machine%20-%20Solomon%20Moulang%20Lewis/Vending%20Machine%20-%20Solomon%20Moulang%20Lewis

Before running the project:

Please read the instructions to install the 'raylib' library onto your computer.

MacOS: https://github.com/raysan5/raylib/wiki/Working-on-macOS
Windows: https://github.com/raysan5/raylib/wiki/Working-on-Windows

File paths from resource folder:

A known issue with using raylib on MacOS Xcode is file paths. Before running the project, you must include the absolute file path for the images and music from the location of the 'resources' folder (included on my GitHub repository) on your computer.

Here is a description of where each of the file locations should be included around the project:

- **Audio.cpp:** The constructor of the audio class provides a file location for the theme music, found in **project folder -> resources -> audio -> vendaway-theme.mp3**

```
themeSong = LoadMusicStream("/Users/solomon-moulang-lewis/Documents/Bath Spa University/CodeLab 1/Github CodeLab 1 -
    Repository/utility-app-SolomonMoulangLewis/Utility App/Vending Machine - Solomon Moulang Lewis/Vending Machine -
    Solomon Moulang Lewis/resources/audio/vendaway-theme.mp3");
```

- **StateSubCategories.hpp:** In the variables section of the class, add file location for background image, found in **project folder -> resources -> textures -> menu_items -> vendaway-display-screen.png**

```
Texture2D background = LoadTexture("/Users/solomon-moulang-lewis/Documents/Bath Spa University/CodeLab 1/Github CodeLab 1 -
    Repository/utility-app-SolomonMoulangLewis/Utility App/Vending Machine - Solomon Moulang Lewis/Vending Machine -
    Solomon Moulang Lewis/resources/textures/menu_items/vendaway-display-screen.png");
```

- **StateMainPage.cpp:** In the constructor definition, add file location for background image, found in: **project folder -> resources -> textures -> menu_items -> vendaway-main-screen-inventory.png**

```
mainPageTexture = LoadTexture("/Users/solomon-moulang-lewis/Documents/Bath Spa University/CodeLab 1/Github CodeLab 1 -
    Repository/utility-app-SolomonMoulangLewis/Utility App/Vending Machine - Solomon Moulang Lewis/Vending Machine -
    Solomon Moulang Lewis/resources/textures/menu_items/vendaway-main-screen-inventory.png");
```

- **StateCategories.cpp:** In the constructor, provide file location found in: **project folder -> resources -> textures -> menu_items -> vendaway-categories.png**

```
//Loads Texture from given file location
catagoriesTexture = LoadTexture("/Users/solomon-moulang-lewis/Documents/Bath Spa University/CodeLab 1/Github CodeLab 1 -
    Repository/utility-app-SolomonMoulangLewis/Utility App/Vending Machine - Solomon Moulang Lewis/Vending Machine -
    Solomon Moulang Lewis/resources/textures/menu_items/vendaway-catagories.png");
```

- **StateCigarettes.hpp:** In the private section of the class, provide file location to the string array for the 6 cigarette images, found in: **project folder -> resources -> textures -> cigs -> chainzsmokez.png/fam-el.png/goldleafdog.png/malbruh.png/ogsmokes.png/puffbox.png**

```
//Stores file location (on my computer) - Not sure how to get general location from project folder
std::string fileLocation[6] =
{{"/Users/solomon-moulang-lewis/Documents/Bath Spa University/CodeLab 1/Github CodeLab 1 -
    Repository/utility-app-SolomonMoulangLewis/Utility App/Vending Machine - Solomon Moulang Lewis/Vending Machine -
    Solomon Moulang Lewis/resources/textures/cigs/chainzsmokez.png"},
{"/Users/solomon-moulang-lewis/Documents/Bath Spa University/CodeLab 1/Github CodeLab 1 -
    Repository/utility-app-SolomonMoulangLewis/Utility App/Vending Machine - Solomon Moulang Lewis/Vending Machine -
    Solomon Moulang Lewis/resources/textures/cigs/fam-el.png"},
{"/Users/solomon-moulang-lewis/Documents/Bath Spa University/CodeLab 1/Github CodeLab 1 -
    Repository/utility-app-SolomonMoulangLewis/Utility App/Vending Machine - Solomon Moulang Lewis/Vending Machine -
    Solomon Moulang Lewis/resources/textures/cigs/goldleafdog.png"},
{"/Users/solomon-moulang-lewis/Documents/Bath Spa University/CodeLab 1/Github CodeLab 1 -
    Repository/utility-app-SolomonMoulangLewis/Utility App/Vending Machine - Solomon Moulang Lewis/Vending Machine -
    Solomon Moulang Lewis/resources/textures/cigs/malbruh.png"},
{"/Users/solomon-moulang-lewis/Documents/Bath Spa University/CodeLab 1/Github CodeLab 1 -
    Repository/utility-app-SolomonMoulangLewis/Utility App/Vending Machine - Solomon Moulang Lewis/Vending Machine -
    Solomon Moulang Lewis/resources/textures/cigs/ogsmokes.png"},
{"/Users/solomon-moulang-lewis/Documents/Bath Spa University/CodeLab 1/Github CodeLab 1 -
    Repository/utility-app-SolomonMoulangLewis/Utility App/Vending Machine - Solomon Moulang Lewis/Vending Machine -
    Solomon Moulang Lewis/resources/textures/cigs/puffbox.png"}
};
```

- **StateBooze.hpp:** In the private section of the class, provide file location to the string array for the 6 drink images, found in: **project folder -> resources -> textures -> drinks -> budwiser.png/kronembro.png/brewdawg.png/rumontheblock.png/whiskeyontheglocks.png/bloodymary.png**

```
std::string fileLocation[6] =
{{"/Users/solomon-moulang-lewis/Documents/Bath Spa University/CodeLab 1/Github CodeLab 1 -
    Repository/utility-app-SolomonMoulangLewis/Utility App/Vending Machine - Solomon Moulang Lewis/Vending Machine -
    Solomon Moulang Lewis/resources/textures/drink/bud-wiser.png"},
{"/Users/solomon-moulang-lewis/Documents/Bath Spa University/CodeLab 1/Github CodeLab 1 -
    Repository/utility-app-SolomonMoulangLewis/Utility App/Vending Machine - Solomon Moulang Lewis/Vending Machine -
    Solomon Moulang Lewis/resources/textures/drink/kronembro.png"},
{"/Users/solomon-moulang-lewis/Documents/Bath Spa University/CodeLab 1/Github CodeLab 1 -
    Repository/utility-app-SolomonMoulangLewis/Utility App/Vending Machine - Solomon Moulang Lewis/Vending Machine -
    Solomon Moulang Lewis/resources/textures/drink/brewdawg.png"},
{"/Users/solomon-moulang-lewis/Documents/Bath Spa University/CodeLab 1/Github CodeLab 1 -
    Repository/utility-app-SolomonMoulangLewis/Utility App/Vending Machine - Solomon Moulang Lewis/Vending Machine -
    Solomon Moulang Lewis/resources/textures/drink/rumontheblock.png"},
{"/Users/solomon-moulang-lewis/Documents/Bath Spa University/CodeLab 1/Github CodeLab 1 -
    Repository/utility-app-SolomonMoulangLewis/Utility App/Vending Machine - Solomon Moulang Lewis/Vending Machine -
    Solomon Moulang Lewis/resources/textures/drink/whiskeyontheglocks.png"},
{"/Users/solomon-moulang-lewis/Documents/Bath Spa University/CodeLab 1/Github CodeLab 1 -
    Repository/utility-app-SolomonMoulangLewis/Utility App/Vending Machine - Solomon Moulang Lewis/Vending Machine -
    Solomon Moulang Lewis/resources/textures/drink/bloodymary.png"}
};
```

- **StateAcessories.hpp:** In the private section of the class, provide file location to the string array for the 6 accessories images (named "drip" in folder), found in: **project folder -> resources -> textures -> drip -> bag.png/calablava.png/chainz.png/scarf.png/grillz.png/tints.png**

```
std::string fileLocation[6] =
{{"/Users/solomon-moulang-lewis/Documents/Bath Spa University/CodeLab 1/Github CodeLab 1 -
    Repository/utility-app-SolomonMoulangLewis/Utility App/Vending Machine - Solomon Moulang Lewis/Vending Machine -
    Solomon Moulang Lewis/resources/textures/drip/bag.png"},
{"/Users/solomon-moulang-lewis/Documents/Bath Spa University/CodeLab 1/Github CodeLab 1 -
    Repository/utility-app-SolomonMoulangLewis/Utility App/Vending Machine - Solomon Moulang Lewis/Vending Machine -
    Solomon Moulang Lewis/resources/textures/drip/calablava.png"},
{"/Users/solomon-moulang-lewis/Documents/Bath Spa University/CodeLab 1/Github CodeLab 1 -
    Repository/utility-app-SolomonMoulangLewis/Utility App/Vending Machine - Solomon Moulang Lewis/Vending Machine -
    Solomon Moulang Lewis/resources/textures/drip/chainz.png"},
{"/Users/solomon-moulang-lewis/Documents/Bath Spa University/CodeLab 1/Github CodeLab 1 -
    Repository/utility-app-SolomonMoulangLewis/Utility App/Vending Machine - Solomon Moulang Lewis/Vending Machine -
    Solomon Moulang Lewis/resources/textures/drip/scarf.png"},
{"/Users/solomon-moulang-lewis/Documents/Bath Spa University/CodeLab 1/Github CodeLab 1 -
    Repository/utility-app-SolomonMoulangLewis/Utility App/Vending Machine - Solomon Moulang Lewis/Vending Machine -
    Solomon Moulang Lewis/resources/textures/drip/grillz.png"},
{"/Users/solomon-moulang-lewis/Documents/Bath Spa University/CodeLab 1/Github CodeLab 1 -
    Repository/utility-app-SolomonMoulangLewis/Utility App/Vending Machine - Solomon Moulang Lewis/Vending Machine -
    Solomon Moulang Lewis/resources/textures/drip/tints.png"}
};
```

- **StateGatsnThat.hpp:** In the private section of the class, provide file location to the string array for the 6 accessories images (named "drip" in folder), found in: **project folder -> resources -> textures -> drip -> duckleknuster.png/glock9.png/revolveglock.png/shank.png/shotglock.png/whackers.png**

```
std::string fileLocation[6] =
{{"/Users/solomon-moulang-lewis/Documents/Bath Spa University/CodeLab 1/Github CodeLab 1 -
    Repository/utility-app-SolomonMoulangLewis/Utility App/Vending Machine - Solomon Moulang Lewis/Vending Machine -
    Solomon Moulang Lewis/resources/textures/shanks/duckleknuster.png"},
{"/Users/solomon-moulang-lewis/Documents/Bath Spa University/CodeLab 1/Github CodeLab 1 -
    Repository/utility-app-SolomonMoulangLewis/Utility App/Vending Machine - Solomon Moulang Lewis/Vending Machine -
    Solomon Moulang Lewis/resources/textures/shanks/glock9.png"},
{"/Users/solomon-moulang-lewis/Documents/Bath Spa University/CodeLab 1/Github CodeLab 1 -
    Repository/utility-app-SolomonMoulangLewis/Utility App/Vending Machine - Solomon Moulang Lewis/Vending Machine -
    Solomon Moulang Lewis/resources/textures/shanks/revolveglock.png"},
{"/Users/solomon-moulang-lewis/Documents/Bath Spa University/CodeLab 1/Github CodeLab 1 -
    Repository/utility-app-SolomonMoulangLewis/Utility App/Vending Machine - Solomon Moulang Lewis/Vending Machine -
    Solomon Moulang Lewis/resources/textures/shanks/shank.png"},
{"/Users/solomon-moulang-lewis/Documents/Bath Spa University/CodeLab 1/Github CodeLab 1 -
    Repository/utility-app-SolomonMoulangLewis/Utility App/Vending Machine - Solomon Moulang Lewis/Vending Machine -
    Solomon Moulang Lewis/resources/textures/shanks/shotglock.png"},
{"/Users/solomon-moulang-lewis/Documents/Bath Spa University/CodeLab 1/Github CodeLab 1 -
    Repository/utility-app-SolomonMoulangLewis/Utility App/Vending Machine - Solomon Moulang Lewis/Vending Machine -
    Solomon Moulang Lewis/resources/textures/shanks/whackers.png"}
};
```

- **StateCart.cpp:** In the class constructor, provide file location to the 'background' variable, found in: **project folder -> resources -> textures -> menu_items -> vendaway-purchase.png**

```
background = LoadTexture("/Users/solomon-moulang-lewis/Documents/Bath Spa University/CodeLab 1/Github CodeLab 1 -
    Repository/utility-app-SolomonMoulangLewis/Utility App/Vending Machine - Solomon Moulang Lewis/Vending Machine -
    Solomon Moulang Lewis/resources/textures/menu_items/vendaway-purchase.png");
```

- **StateUserInventory.cpp:** In the class constructor, provide file location to the 'background' variable, found in: **project folder -> resources -> textures -> menu_items -> vendaway-inventory.png**

```
background = LoadTexture("/Users/solomon-moulang-lewis/Documents/Bath Spa University/CodeLab 1/Github CodeLab 1 -
    Repository/utility-app-SolomonMoulangLewis/Utility App/Vending Machine - Solomon Moulang Lewis/Vending Machine -
    Solomon Moulang Lewis/resources/textures/menu_items/vendaway-inventory.png");
```

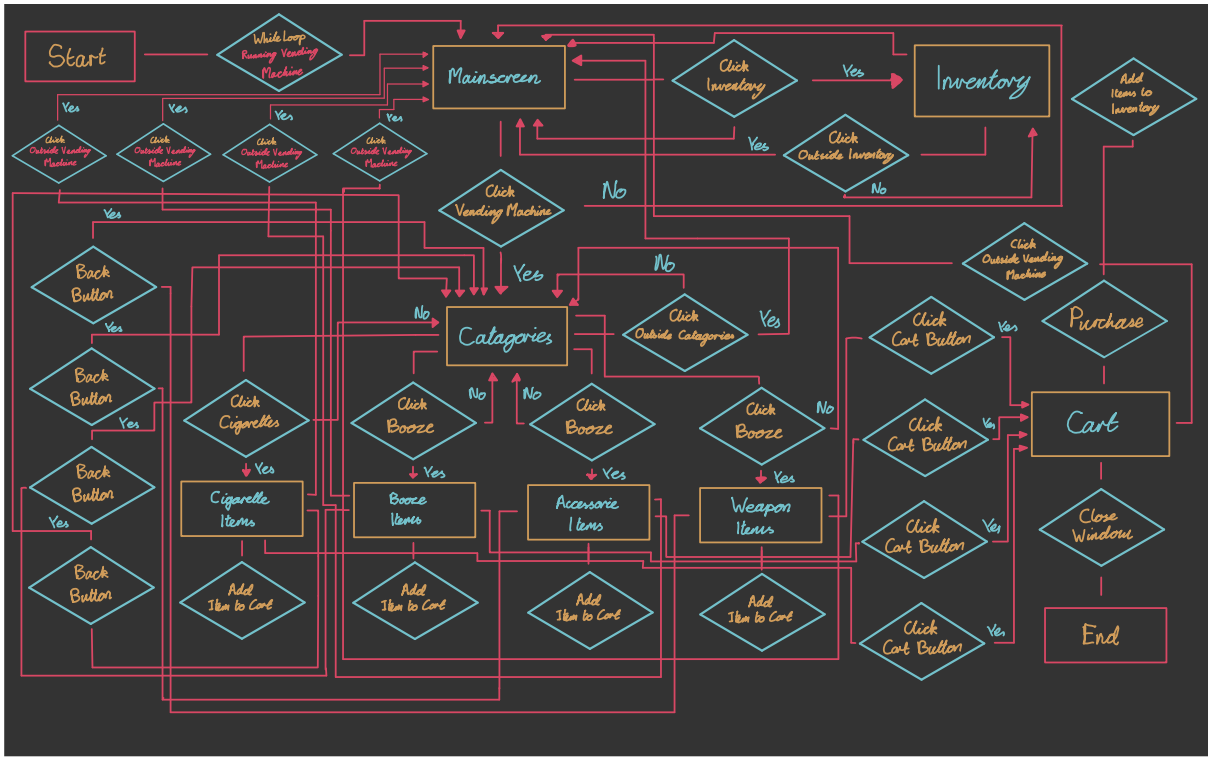
Specification

- *Main page:* **Displays:** Main page image of the vending machine
  Actions: to open **Categories**
  Displays: an image of the user
  Actions: to open: **User Inventory**
- *Item category page:* **Displays:** four categories
  Actions: to move to the new page of each sub-category.
  Displays: cart button
  Actions: move to the cart page.
- *Sub-category pages x4:* **Displays:** Categories background image
  Displays: 6 items per page (four different pages)
  Displays: 'items', 'item stock', 'item price' and 'item name' per item
  Actions: 'add item to cart (with a mouse click)'
  Displays: 'back button' and 'cart button'
  Actions: jump to the specified page
- *User Inventory page:* **Displays:** list of items the user has bought (not fully working)
- Cart page: **Displays**: Cart background Image
  Displays: 'user's money', 'input money', 'total cost', 'change'
  Actions: input money +/- $5, purchase items
  Displays: 'back button'
  Actions: jump to cart state page

- A way of capturing the **mouse position** and **mouse click** of a user.
  Used to 'switch pages', 'add items to cart', 'input money' and 'purchase items'
- A way of **keeping track of 'item stock'**
- A way of **keeping track of the 'user's money'**
- A way of **inputting money**
- A way of **keeping track of the user's change** from a transaction and **giving them back the change from the transaction**

System Flowchart:

I have created a flowchart showing the whole process of the vending machine application, it includes the processes decisions and inputs/outputs that the program contains:



Project Planning:

For this program I wanted to do something I hadn't tried before. I had been starting to learn how libraries work in the weeks leading up to the Vending Machine Set Date and I decided it would be a good thing for me to learn how to do.

I chose the 'raylib' library to use in my project as I liked the simplicity of the classes within the library and the classes and method descriptions clear. https://www.raylib.com

I first followed a video by a YouTube channel 'Yearning' called "C++ Game Programming Tetris Raylib". Which is a four-part tutorial taking you through the steps of creating a Tetris game with the raylib library. This tutorial quickly became out of my depths although I gained a lot of useful information about using the raylib library itself.
https://www.youtube.com/channel/UCZi2pchtDxk12Oz4h9pKFBg

I liked the way how 'Yearning' implemented the raylib functions needed to create a main window into a new file with a class. I used a similar technique when implementing this into my own project.

I spent some time going through the raylib library 'cheat sheet' to see what classes and methods I would need to implement in my own project. https://www.raylib.com/cheatsheet/cheatsheet.html

I then followed the instructions through GitHub to install the library on my computer:
https://github.com/raysan5/raylib/wiki/Working-on-macOS
This also contained instructions for setting up a project in Xcode with the raylib library.

I then spent days messing about with the raylib library creating various projects until I was content with using it within my vending machine project.

In my spare time I started creating pixel art images on an application on my iPad. I started creating sketches of a world which could fit around the vending machine.
I had been to Spain previously in the year and this vending machine project reminded me of the vending machines near where I stayed in Seville. This image of dishevelled back-alley streets, broken bottles and cigarette butts surrounded by a gleaming warm city became my inspiration for this project.

Technical Description:

Brief summary of the project layout of my application:

Key: Class, .cpp, .hpp

- **States:** Represent the different states or 'pages' of my application
  - **State Main-Page:** Class - .cpp and .hpp files
  - **State Inventory:** Class - .cpp and .hpp files
  - **State Categories:** Class - .cpp and .hpp files
  - **State Sub-Categories:** Class - Used as a template - .cpp and .hpp files
  - **State Cigarettes:** Class - Inherits from Sub-Categories - .cpp & .hpp file
  - **State Booze:** Class - Inherits from Sub-Categories - .cpp & .hpp file
  - **State Accessories:** Class - Inherits from Sub-Categories - .cpp & .hpp file
  - **State GatsnThat:** Class - Inherits from Sub-Categories - .cpp & .hpp file
  - **State User Inventory:** Class - .cpp and .hpp files
- **Inventory:** Class - .cpp and .hpp files
  - Stores and tracks inventory and item cost
- **Implementation:** Class - .cpp and .hpp files
  - Stores instances of all 'states'
  - Implements the functionality of the program
- **Main Window:** Class - in .cpp and .hpp files
  - Stores the functions to create a window in raylib
- **Button:** Class - .cpp and .hpp files
  - Object I created that generates a button
  - Various methods
  - Used throughout the program
- **Audio:** Class - .cpp and .hpp files
  - Loads and unloads music file from file location (on my computer) to RAM.
  - Update function (raylib), keeps music playing

Summary of Main Window Class:

When creating a window with the raylib library there are multiple functions that must be called. As the YouTuber Yearning states in his video “we are just calling some free functions that the library defines,” (Yearning, 2021, 00:01:24 – 00:01:29) and there is nothing preventing us from not calling these functions. If any function is missed the window will not open properly.
To make sure no function call is missed Yearning places these functions inside a class within a new .cpp and header file.
For my project I followed a similar process creating a new header and cpp file containing a class called ‘MainWindow’. My constructor and destructor initialise and close the window, so I don’t have to worry about calling these functions. From my main cpp file I can then call the individual functions when needed, like so:

```
MainWindow mainWindow;
```

```
//Application Loop
while (!mainWindow.AppShouldClose())
{
    //------------------------------------------
    //Update
    themeSong.Update();
    //------------------------------------------
    //Draw
    mainWindow.Draw();

    implementation.SwitchStates();

    mainWindow.EndDraw();
    //------------------------------------------
}
```

Summary of Audio Class:

I decided to include some music in my project, so I spent an afternoon creating a track for the vending machine.
I created new header and cpp files containing a class called ‘Audio’ and used raylib library functions to load and play the audio file.
The constructor and destructor load and unload the audio file from RAM so I don’t need to call these from my main file.
The update function then includes the functions to update the audio buffer and play the audio file:

```
//-----------------------------------------------
//Updates every frame for consistent audio
void Audio::Update()
{
    //Play Music
    UpdateMusicStream(themeSong);
    PlayMusicStream(themeSong);
}
//-----------------------------------------------
```

So, from my main file, all I need to do is instantiate the class and call the update function in the main while loop:

```
    Audio themeSong;
    Implementation implementation;

    //Initialise all States
    implementation.InitStates();

//-----------------------------------------------
    //Application Loop
    while (!mainWindow.AppShouldClose())
    {
        //-----------------------------------
        //Update
        themeSong.Update();
        //-----------------------------------
        //Draw
        mainWindow.Draw();

        implementation.SwitchStates();

        mainWindow.EndDraw();
        //-----------------------------------
    }
```

Summary of Button Class:

I began to think of a way to start implementing my program and decided to create a button object which could be used to do different functions based on the user's mouse position and mouse click.
I created a new class in new cpp and header files.
I gave the button constructor arguments to change its position and size on the screen whenever the button class is instantiated:

```
Button(float posX, float posY, float width, float height);
```

A draw method to change the colour of the button border when the mouse is hovered over:

```
//Draws Rectangle as border
void Button::Draw(Color outline, Color highlight)
{
    //If mouse hovered, border appears RED, else BLANK
    if (CheckCollisionPointRec(GetMousePosition(), buttonBorder))
        DrawRectangleLines(buttonBorder.x, buttonBorder.y, buttonBorder.width, buttonBorder.height, highlight);
    else
        DrawRectangleLines(buttonBorder.x, buttonBorder.y, buttonBorder.width, buttonBorder.height, outline);
}
//-----------------------------------------------------------------------------------------------------------
```

And boolean functions that return true for a frame when the mouse is clicked on or off the button.

```
//Returns true if Mouse Clicked inside button
bool Button::MouseClicked()
{
    if (IsMouseButtonPressed(MOUSE_BUTTON_LEFT) && CheckCollisionPointRec(GetMousePosition(), buttonBorder))
    {
        return true;
    }
    return false;
}
//-----------------------------------------------------------------------------------------------------------
//Returns true if mouse clicked outside button
bool Button::MouseClickedOut()
{
    if (IsMouseButtonPressed(MOUSE_BUTTON_LEFT) && !CheckCollisionPointRec(GetMousePosition(), buttonBorder))
    {
        return true;
    }
    return false;
}
```

Summary of Inventory Class:

I created a class to track the stock inventory of the vending machine.
Each sub-category contains six items, so I created an array of integers to represent the stock size of each item:

```
//Class used to track the stock and price of the items
class Inventory
{
public:
    //Cigarettes Stock
    int cigaretteStock[6]       = {1, 3, 1, 5, 1, 2};
    std::string cigarettePrice[6]    = {"03.99", "04.50", "03.45", "06.10", "05.99", "07.50"};

    //Booze Stock
    int boozeStock[6]           = {1, 3, 3, 2, 3, 1};
    std::string boozePrice[6]        = {"03.41", "04.60", "02.80", "05.98", "06.40", "08.99"};

    //Accessories Stock
    int accessorieStock[6]      = {3, 2, 1, 3, 1, 2};
    std::string accessoriePrice[6]   = {"49.99", "05.98", "35.40", "03.49", "100.0", "99.95"};

    //Gats n That
    int gatsnThatStock[6]       = {1, 2, 1, 2, 2, 4};
    std::string gatsnThatPrice[6]    = {"25.99", "105.0", "249.0", "50.99", "433.0", "40.00"};
```

I also included a function to reset the stock of each of the items:

```
//Initialises the stock system
void InitInventory()
{
    //Cigarette stock Init
    cigaretteStock[0] = 1;
    cigaretteStock[1] = 3;
    cigaretteStock[2] = 1;
    cigaretteStock[3] = 5;
    cigaretteStock[4] = 1;
    cigaretteStock[5] = 2;

    //Booze Stock Init
    boozeStock[0] = 1;
    boozeStock[1] = 3;
    boozeStock[2] = 3;
    boozeStock[3] = 2;
    boozeStock[4] = 3;
    boozeStock[5] = 1;

    //Accessorie Stock Init
    accessorieStock[0] = 3;
    accessorieStock[1] = 2;
    accessorieStock[2] = 1;
    accessorieStock[3] = 3;
    accessorieStock[4] = 1;
    accessorieStock[5] = 2;

    //Gats n That Stock Init
    gatsnThatStock[0] = 1;
    gatsnThatStock[1] = 2;
    gatsnThatStock[2] = 1;
    gatsnThatStock[3] = 2;
    gatsnThatStock[4] = 2;
    gatsnThatStock[5] = 4;
}
```

Summary of Implementation File

The implementation class is the heart of my vending machine application, it defines how the program runs by implementing all the object classes I have added this project.
I have created a variety of different classes included in header and .cpp files throughout the program. I split the functionality of the program into 'states', where each state can be considered as its own page and is represented using a class. The implementation file then includes all these states and calls the methods within the classes to create the functionality of the program.

Include Classes

I first included all the classes I had created into the implementation .hpp file by using the #include keyword to locate the .hpp files:

```
#include "StateMainPage.hpp"
#include "StateCatagories.hpp"
#include "StateCigarettes.hpp"
#include "StateBooze.hpp"
#include "StateAccessories.hpp"
#include "StateGatsnThat.hpp"
#include "StateCart.hpp"
#include "Inventory.hpp"
#include "MainWindow.hpp"
#include "StateUserInventory.hpp"
```

I then created a class named 'implementation' in the header .hpp file and used the private section of the class to instantiate all my classes:

```
private:
    //Creating instances of each state
    StateMainPage mainPage;
    StateCatagories catagories;
    StateCigarettes cigarettes;
    StateBooze booze;
    StateAccessories accessories;
    StateGatsnThat gatsnThat;
    StateCart cart;
    StateUserInventory userInventory;
    Inventory inventory;
```

Methods and Functions

I use multiple methods included in the implementation class to create the functionality of the vending machine.

Firstly, I created a function declaration to initialise variables in the classes included:

```
void InitStates();
```

I then define the function in the .cpp file and call the functions to initialise the classes where need be:

```
void Implementation::InitStates()
{
    //Loads textures into GPU (VRAM) for each sub-category state
    cigarettes.LoadTextures();
    booze.LoadTextures();
    accessories.LoadTextures();
    gatsnThat.LoadTextures();

    inventory.InitInventory();//Inventory Init
}
```

States

I created an enum in the public section of my class is used to store the names of each of the states:

```
public:
    //Stores all the project application state names
    enum States {
        MAINPAGE,
        CATAGORIES,
        CIGARETTES,
        BOOZE,
        ACCESSORIES,
        GATSNTHAT,
        CART,
        USERINVENTORY,
    };
```

The DrawState function takes the enum ‘States’ as an argument:

```
void DrawState(States states);//Draws states
```

For the functions definition in the .cpp file I create a switch case statement. Each ‘case’ will call the draw functions of each state and any additional methods needed:

```
//Switch Statement calls Draw Functions for each state + Processes need for each state
void Implementation::DrawState(States states)
{
    switch (states) {
        case States::MAINPAGE:
            mainPage.Draw();
            break;
        case States::CATAGORIES:
            catagories.Draw();
            break;
        case States::CIGARETTES:
            cigarettes.Draw();
            cigarettes.DrawItemText();
            cigarettes.DrawItemPrice();
            DrawCigaretteItemStock();
            AddCigaretteItemToCart();
            DrawCigaretteItemOutOfStock();
            break;
        case States::BOOZE:
            booze.Draw();
            booze.DrawItemText();
            booze.DrawItemPrice();
            DrawBoozeItemStock();
            AddBoozeItemToCart();
            DrawBoozeItemOutOfStock();
            break;
        case States::ACCESSORIES:
            accessories.Draw();
            accessories.DrawItemText();
            accessories.DrawItemPrice();
            DrawAccessorieItemStock();
            AddAccessorieItemToCart();
            DrawAccessorieItemOutOfStock();
            break;
        case States::GATSNTHAT:
            gatsnThat.Draw();
            gatsnThat.DrawItemText();
            gatsnThat.DrawItemPrice();
            DrawGatsnThatItemStock();
            AddGatsnThatItemToCart();
            DrawGatsnThatItemOutOfStock();
            break;
        case States::CART:
            cart.Draw();
            ProcessClearButton();
            cart.PurchaseButtonProcess();
            break;
        case States::USERINVENTORY:
            userInventory.Draw();
    }
}
```

This allows me to call a simple and clear function to draw each state whenever needed:

```
DrawState(States::ACCESSORIES);
```

Switch States

For the next step of my program, I declare a function ‘switch states’:

```
void SwitchStates(); //Used to switch states based on conditions
```

This method will be the core functionality of my program defining how the program switches states, adds items and clears items in the cart.

However, before we look into this method we must first look at the other methods in the class.

I was struggling with how I could switch the states of the class to show a new page of the vending machine. A state should switch depending on a mouse click on the current screen, for example the ‘Main screen’ switches to the ‘Category screen’ when the mouse of the user clicks on the image of the vending machine.
Looking at the **StateMainPage** class:
I created a method in the class that returns a boolean value when the user’s mouse clicks on a certain object. This class has two of these methods, one returns true when the mouse clicks on the image of the vending machine, the other returning true when mouse clicks on the inventory button. However, both these methods only return true for a frame, so as soon as that frame passes the

methods return false again. To combat this, I created boolean variables in the private section of the Implementation class. The boolean methods from the state classes then toggle these boolean variables in the implementation file. The variables hold their value, meaning I can check them in an if-statement and use them to hold the current state.
To do this we can look at the **StateMainPage** class. All other classes then use very similar functionality.
I used the Button class I had made to draw a rectangle around the vending machine. The button has a method that returns true when the mouse has clicked on that rectangle:

```
bool Button::MouseClicked()
{
    if (IsMouseButtonPressed(MOUSE_BUTTON_LEFT) && CheckCollisionPointRec(GetMousePosition(), buttonBorder))
    {
        return true;
    }
    return false;
}
```

The class is then instantiated in the **StateMainPage** class and another boolean method inside the class is used to return true when the mouse clicks the button belonging to that class:

```
bool StateMainPage::MouseClickedButtonMain()
{
    return buttonMain.MouseClicked();
}
```

Then in the Implementation methods are used to check button presses from multiple classes by checking the boolean methods inside the state classes using an if-statement. The boolean variables of the implementation class are then updated accordingly:

```
bool Implementation::MainPageToggler()
{
    if (catagories.MouseClickedButtonBorder()   ||
        cigarettes.MouseClickedBorder()         ||
        booze.MouseClickedBorder()              ||
        accessories.MouseClickedBorder()        ||
        gatsnThat.MouseClickedBorder()          ||
        userInventory.MouseClickedBorder()      )
    {
        mainPageToggle      = true;
        catagoriesToggle    = false;
        cigarettesToggle    = false;
        boozeToggle         = false;
        accessoriesToggle   = false;
        gatsnThatToggle     = false;
        cartToggle          = false;
        userInventoryToggle = false;
        return true;
    }
    return false;
}
```

Now in the SwitchStates method of the Implementation file an if-statement checks the return type of each of the boolean methods inside the class and calls the methods inside the states to update and display onto the window:

```
//Switch States and Draw States
void Implementation::SwitchStates()
{
    MainPageToggler();  //Check main page toggle if true -> goes to 'else' -> Draw MainPage();
    if (catagoriesToggle) { //If Categories Toggle then:
        DrawState(States::CATAGORIES);
        CigarettesToggler();
        BoozeToggler();
        AccessoriesToggler();
        GatsnThatToggler();
        CartToggler();
    }
    else if (cigarettesToggle)  { //If Cigarettes Toggle then:
        DrawState(States::CIGARETTES);
        CartToggler();
        if (cigarettes.MouseClickedBackButton())        goto End; //Jumps to Marker
        if (cigarettes.MouseClickedCartButton())        goto Cart; //Jumps to marker
    }
    else if (boozeToggle) { //If Booze Toggle then:
        DrawState(States::BOOZE);
        CartToggler();
        if (booze.MouseClickedBackButton())             goto End; //Jumps to Marker
        if (booze.MouseClickedCartButton())             goto Cart;//Jumps to Marker
    }
    else if (accessoriesToggle) { //If Accessories Toggle then:
        DrawState(States::ACCESSORIES);
        CartToggler();
        if (accessories.MouseClickedBackButton())       goto End;//Jumps to Marker
        if (accessories.MouseClickedCartButton())       goto Cart;//Jumps to Marker
    }
    else if (gatsnThatToggle) { //If GatsnThat Toggle then:
        DrawState(States::GATSNTHAT);
        CartToggler();
        if (gatsnThat.MouseClickedBackButton())         goto End;//Jumps to Marker
        if (gatsnThat.MouseClickedCartButton())         goto Cart;//Jumps to Marker
    }
    else if (cartToggle) {  //If Cart Toggle then:
        Cart:
        DrawState(States::CART);
        if (cart.MouseClickedBackButton())              goto End;//Jumps to Marker
    }
    else if (userInventoryToggle) { //If User Inventory Toggle then:
        DrawState(States::USERINVENTORY);
        MainPageToggler();
    }
    else {  //Else Draw MainPage()
        End:
        DrawState(States::MAINPAGE);
        UserInventoryToggler();
        CatagoriesToggler();
    }
}
```

Adding items to cart

Depending on the item category state, an item should be added to the cart when a user clicks the item. To do this a similar method layout is used for each item category state. I added methods in the implementation file for each of the sub item categories, ‘cigarettes’, ‘booze’, ‘accessories’, ‘gatsnThat’. Each method uses a **for loop** to check whether one of the six items has been clicked. If an item has been clicked then the function does one of three things.

1. Adds the total cost of an item to a variable in the cart class.
2. Decreases the stock quantity of the item by 1.
3. Adds 1 to the item quantity variable in the cart class.

```
void Implementation::AddCigaretteItemToCart()
{
    for (int i = 0; i < 6; i++)
    {
        if (cigarettes.button[i].MouseClicked() && inventory.cigaretteStock[i] > 0)
        {
            if (cart.itemQuantity <= cart.maxItems)
            {
                inventory.cigaretteStock[i]--;
                cart.CalculateTotalCost(cart.StringToFloat(inventory.cigarettePrice[i]));
                cart.itemQuantity++;
            }
        }
    }
}
```

These methods are then checked in each case of the switch-case statement inside the DrawState method. I did this in an attempt to save on CPU usage so that the **for loop** only runs depending on the particular case.

Draw Stock

Methods inside the Implementation class are used to draw the stock quantity of an item:
The method calls another method from one of the sub-category item states and draws it to screen.

```
void Implementation::DrawCigaretteItemStock()
{
    cigarettes.DrawItemInventoryQuantity(inventory.cigaretteStock, cigarettes.itemInventoryBox);
}
```

Draw item out of stock sign

I added methods inside each of the sub-category item state classes that draw over the item image to display an out-of-stock sign:

```
void Implementation::DrawCigaretteItemOutOfStock()
{
    for (int i = 0; i < 6; i++)
    {
        if (inventory.cigaretteStock[i] < 1) cigarettes.DrawOutOfStockSign(i);
    }
}
```

User Inventory

I spent hours trying to figure out a way to display the items the user had bought in the **StateUserInventory.** I tried multiple different ways, unfortunately none of which worked. I first tried to pass the item file locations from the string arrays inside each of the sub-category item state classes, to a new string array in the **StateUserInventory** class. This didn't work as I kept getting a BAD_ACCESS_THREAD error. I tried adding all the item file locations to variables in the **StateUserInventory** class and displaying them when an item was pressed. This didn't work either as weird things started happening visually on the screen.
I had to give up in the end as I was running out of time, but I hope to tackle this problem later in the future.

Critical Reflection

Before creating this application, I had never used an external library before with my code. I started the application early on because I wasn't sure whether I would be able to understand how to use the library. However, the Raylib library was clear to use after some research, and the documentation on the website was easy to read and understand. Having spent a lot of time playing around with the library and trying and testing different things it eventually became apparent that this was an achievable goal.

I am very pleased with the outcome of my work. I think the graphical user interface is smooth and easy to understand, for the most part. The user experience is an essential part of an application, and it is worth spending time to create a smooth enjoyable experience. Although I think I did this for the most part, it became apparent when showing my application to friends and family that I could have made elements of the app clearer to the user.

Although I think I did well with what I have made, I know I that there is a lot I could have improved on. This became apparent when I started adding more things to the program. When one part of my program was built on a slightly questionable design, it would mean the rest had to fit into that design, adding a lot of unnecessary code and work for myself. There is a lot of repeated code, especially in the implementation file that could have been cut down with better design. There are bugs within the program which make it crash sometimes, and because of the overuse of code in certain areas in my design, I found it hard to backtrack to where the bugs were coming from. The implementation is arguably the worst part of my coding in this project, containing a lot of repeating code and questionable design flaws. One of the worst being the way the program switches states. I from research that using the keyword 'goto' is bad design for any c++ program. I should have implemented another way to do this.
I discovered a method of programming after I had finished the vending machine that would have been extremely useful for my application. The state machine design pattern makes it easy to switch program states, it also free's up memory making the program run more efficiently.

Overall, I am proud out what I have created, and I have learnt a lot of useful skills with programming and using libraries.

References:

Yearning (2021): C++ Game Programming Tetris Raylib – Part 1 [Presentation & Setup]. [online video] Available at: <https://www.youtube.com/watch?v=gO6tuky1zNg&t=244s> [Accessed: 22 December 2021].